

Proof By Message Passing

Mark Oussoren
Georgia Tech
moussoren3@gatech.edu

Benjamin Chen
Georgia Tech
bchen483@gatech.edu

Diego Franco
Georgia Tech
dfranco32@gatech.edu

Rahul Palnitkar
Georgia Tech
rpalnitkar3@gatech.edu

Abstract

*We trained several neural networks to automate the writing of formal mathematical proofs, using Lean 4 as the verification system. Our pipeline includes a graph neural network (GNN) for structured proof-state encoding, an encoder-decoder Transformer for tactic sequence generation, a second Transformer for translating formal proofs into readable prose, and an AST-based GNN for structured tactic prediction with Lean kernel feedback. The Transformer decoder model proved 6 of 50 held-out theorems; an extended argmax rollout proved 12 of 126 Mathlib theorems with 46 accepted proof steps.*¹

1. Introduction

1.1. What Did We Try to Do?

A *formal proof* is a mathematical argument written in a programming language that a computer can check for correctness, line by line, without human judgment. Our goal is to teach a neural network to write such proofs automatically. Concretely, the network is given an unproved mathematical statement and must output a sequence of *proof steps* that a verification engine accepts as a complete, correct argument. We used Lean 4 as the verification engine because it has the largest publicly available library of formalized mathematics.

We pursued four objectives: (1) replicate the GNN theorem-proving agent of [3], which encodes proof states as graphs and predicts proof steps from a fixed vocabulary of six operations; (2) build a Transformer that frames proof-step prediction as machine translation; (3) build a second Transformer that converts formal Lean proofs into natural mathematical prose (autoinformalization [24]); and (4) design a structured tactic-prediction pipeline with direct Lean

kernel feedback. The contributions of each member is outlined in Table 4.

1.2. How Is It Done Today?

Lean is a theorem prover and programming language designed for writing and verifying mathematical proofs [8]. Mathlib is a community library of formalized mathematics written in Lean 4 [18]. Proofs in Lean are written using *tactics*: each tactic transforms the current proof obligation into zero or more simpler subgoals, and a proof is complete when all obligations are discharged [8]. The central difficulty for learned provers is that the space of applicable tactics at any state is effectively unbounded.

Nazrin [3] addresses this by restricting to six atomic tactics claimed to suffice for any provable Lean theorem, and trains a GNN on resulting proof-state graphs. LLM-based provers such as ReProver [25] operate over full tactic strings and achieve strong results but require five days of training on an NVIDIA A100, placing them beyond a student compute budget. Autoinformalization is currently driven by large language models [24]; earlier work used smaller neural networks [21].

1.3. Who Cares?

Mathematics is verified today by human peer review, a process that is slow and occasionally wrong. Automated formal verification would allow mathematical results to be checked mechanically in seconds. If learned provers can handle research-level problems such as Putnam contest questions, the bottleneck shifts from *checking* proofs to *posing* interesting questions, which is a fundamentally different and more creative task. Progress on formal proof automation also provides a concrete, objectively graded benchmark for measuring mathematical reasoning in neural networks, analogous to the role chess engines played in evaluating combinatorial search algorithms.

¹The project repository can be found [here](#).

1.4. Data

1.4.1 Mathlib v4.29.0

We re-implemented the transposing atomization algorithms of [3] to convert $n = 285,752$ out of 522,403 ($\approx 55\%$) Mathlib theorems into ExprGraph format (the paper reports 58%; the gap reflects custom tactic modifications we could not replicate without the authors’ source code). The tactic distribution is heavily skewed: `intro` appears 276,005 times while `revert` appears only twice (Table 6). This imbalance directly motivated our choice of focal loss for tactic-kind classification. The dataset is intended for tactic prediction and proof-state encoding only.

There are several limitations with the data. For one, it inherits the mathematical subfield distribution of Mathlib, which skews toward algebra and topology; coverage of analysis and combinatorics is sparser and models trained on it may not generalize uniformly across mathematical domains. Additionally, only 55% of Mathlib theorems were successfully atomized, and the six-tactic vocabulary may not cover all proof patterns in the remaining 45%. Lastly, the `cases` tactic is severely underrepresented (6,635 examples), making reliable argument prediction for branching proofs infeasible with this corpus alone.

1.4.2 LeanDojo Benchmark

Training data for the Transformer consisted of $\sim 250k$ state-tactic pairs from the LeanDojo Benchmark 4 dataset [5, 25]; validation and test sets contain 4,260 and 4,510 entries respectively. Both inputs (goal states) and outputs (tactics) are strings of Lean expressions. A custom byte-pair encoding (BPE) tokenizer was trained on the combined vocabulary of goal states and tactics in the training split, with a vocabulary size of 16,000 tokens, to handle Lean’s mathematical Unicode characters and identifier naming conventions. An example state-tactic pair is shown in Table 5. The data were collected by tracing Lean 4 proofs from Mathlib using the LeanDojo toolkit, which instruments the Lean kernel to record every intermediate goal state and the tactic that advanced it [25]. No human annotation beyond the original Mathlib proofs is involved. The main usage of the data is for tactic prediction: given a goal state string, predict a syntactically valid Lean tactic that advances the proof.

There are several limitations with the data with label noise being the primary concern as the same goal state can be advanced by many distinct tactics, yet only the tactic actually used in the traced proof is recorded as ground truth. Exact-match accuracy is therefore a strict lower bound on model quality. The output vocabulary is open-ended because Lean tactics can reference arbitrary Mathlib identifiers, motivating sequence generation rather than classification.

1.4.3 Herald

The autoinformalization Transformer uses the Herald dataset [9], derived from Mathlib 4 with a hierarchical retrieval-augmented annotation [22] procedure supervised by PhD mathematicians. The training split contains $\sim 44k$ aligned pairs of Lean statements and natural-language proofs. Labels were generated by prompting large language models [7] [6] with retrieved Mathlib context and then verified by human experts. The dataset is intended for translating formal Lean proofs into readable mathematical prose.

The main limitation here worth noting is that label quality is bounded by the LLM used for annotation. Proofs requiring deep domain knowledge may be paraphrased incorrectly in the reference translations.

2. Approach

2.1. Nazrin GNN

Because [3] provides no implementation details and the authors declined to share source code, we designed our own GNN referencing the graph learning literature [26, 11].

Each node v of the ExprGraph receives a feature vector $\mathbf{x}_v = [\mathbf{t}_v \parallel \mathbf{f}_v]$, where $\mathbf{t}_v \in \{0, 1\}^{|S|}$ is a one-hot encoding of the syntactic category (e.g., APP, LAM, CONST) and $\mathbf{f}_v \in [0, 1]^{d_e}$ is a feature-hash embedding of the node’s string label. Feature hashing maps a string to a fixed-length vector by applying a hash function to each character n -gram and accumulating signed unit contributions at the corresponding indices; no learned embedding table is required, which is advantageous when the identifier vocabulary is unbounded. Nodes are projected to hidden dimension d via GELU [10]:

$$\mathbf{h}_v^{(0)} = \text{GELU}(W_{\text{in}}\mathbf{x}_v + \mathbf{b}_{\text{in}}), \quad \text{GELU}(x) = x\Phi(x).$$

For layer $\ell \in \{1, \dots, L\}$, denoting $\mathcal{N}(v)$ as in-neighbors:

$$\mathbf{a}_v^\ell = \frac{\mathbf{h}_v^{\ell-1} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{\ell-1}}{1 + |\mathcal{N}(v)|},$$

$$\mathbf{h}_v^\ell = \mathbf{h}_v^{\ell-1} + \text{GELU}(W^\ell \mathbf{a}_v^\ell + \mathbf{b}^\ell).$$

LayerNorm [4] is applied to final embeddings to reduce training time: $\mathbf{H}_v = \text{LayerNorm}(\mathbf{h}_v^L)$.

Three heads are trained jointly: (1) a tactic-kind softmax classifier $\hat{\mathbf{p}}_v = \text{softmax}(W\mathbf{H}_v)$; (2) a bilinear pointer network for argument-node selection with scaled dot-product scores $s_{uv} = (W_Q\mathbf{H}_v)^\top (W_K\mathbf{H}_u) / \sqrt{d}$; and (3) a graph-level cross-section MLP via mean pooling. The joint loss is

$$\lambda_\tau \mathcal{L}_{\text{Focal}}(\hat{\mathbf{p}}_v, y_v) + \lambda_{\text{arg}} \mathcal{L}_{\text{CE}}(\mathbf{s}_v, a_v) + \lambda_{\text{cs}} \mathcal{L}_{\text{MSE}}(\hat{c}, c).$$

Focal loss [13] is used for tactic-kind classification to down-weight the majority classes (`intro`, `apply`) and improve gradient signal for rare tactics.

We expected the extreme class imbalance (Table 6) to dominate training and the 6,635 `cases` examples to be insufficient for reliable branching-argument prediction. Both problems materialized. We did not anticipate that the six-tactic completeness claim would be difficult to verify empirically, nor that mean aggregation would cause oversmoothing severe enough to prevent tactic-kind separation.

2.2. Transformer Tactic Generator

We framed tactic generation as sequence-to-sequence machine translation: the goal state string is the source and the tactic string is the target. Both share vocabulary (identifier names, type constructors, punctuation) but differ in grammatical role. We used a standard encoder-decoder Transformer [20] with sinusoidal positional encodings. The BPE tokenizer described in Section 1.4.2 encodes inputs; the decoder generates tactics autoregressively. The training objective is cross-entropy over the target sequence:

$$\mathcal{L}_{\text{CE}} = - \sum_{j=1}^m \log p_{\theta}(y_j \mid y_{<j}, \mathbf{x}).$$

Teacher forcing is applied during training to stabilize early gradient flow. Cross-attention in the decoder allows each output token to attend over the full encoded goal state, implicitly learning which hypotheses are relevant for the next tactic token. Hyperparameters were tuned via Optuna [1] using a Tree-structured Parzen Estimator (TPE) over 10 trials of 3 epochs each. The optimal configuration is 4 encoder and 4 decoder layers, hidden dimension $d_{\text{model}} = 256$, $H = 4$ attention heads, feedforward dimension $d_{\text{ff}} = 1,024$, and AdamW with learning rate $\eta = 2.30 \times 10^{-4}$ and weight decay $\lambda = 0.0552$ (Table 8).

We anticipated that the open-ended tactic vocabulary would make exact-match accuracy low and that 250k state-tactic pairs might be insufficient relative to ReProver’s training regime. We did not anticipate the severity of exposure bias [23]: at inference time the decoder receives its own (incorrect) predictions rather than ground-truth prefixes, causing errors to compound and producing repetitive, degenerate outputs.

2.3. Transformer Autoinformalization

While current state-of-the-art methods use large language models [24], more recent architectural advances motivate their application to informalization. We incorporate feed-forward expansion [20], Rotary Positional Embeddings (RoPE) [17], and looped Transformers [16]. Looped Transformers pass the output of a layer back into its input for multiple iterations, matching the performance of deeper models on mathematical reasoning tasks while using fewer parameters. RoPE encodes relative position multiplicatively in the attention weights and generalizes better

to proof lengths not seen during training than sinusoidal encodings. Feed-forward expansion enlarges the hidden dimension of each feed-forward sublayer to a multiple of the embedding dimension, increasing the model’s capacity to memorize Lean-specific identifier patterns. We use focal loss and label smoothing of 0.1 to address the class imbalance in tactic identifiers.

We expected BLEU to be a noisy metric for mathematical prose because paraphrases of a correct proof can differ arbitrarily in surface form. This was confirmed: early runs produced repetitious run-on sentences with high n-gram overlap but no mathematical coherence. We did not anticipate that the looped architecture, despite being smaller, would overfit more slowly than the full-depth baseline.

2.4. Structured Prediction

We built a series of models via phases from tactic prediction (simple) to models capable of proving full theorems. All models output a tactic kind head and n argument heads where both heads share the prediction vector. An argument can be local (defined only in the Lean theorem context) or global (defined in Mathlib elsewhere). An argument head computes $z \rightarrow \text{MLP} \rightarrow \text{softmax} \rightarrow \arg \max$ over $[\text{EMPTY}, \text{LOCAL}_0, \dots, \text{GLOBAL}]$, where `LOCAL` is the hypothesis position in the Lean theorem. When `GLOBAL` is chosen, the string is chosen from retrieving the nearest normalized global embedding from training (cosine similarity). When `EMPTY` is chosen it means no argument is rendered.

Model Alias	Description
<code>mlp.narg</code>	AST+GNN goal embedding g . <code>narg</code> means n argument heads.
<code>cross.attn</code>	g and local hypothesis position embeddings with one goal cross-attention.
<code>cross.attn.concat</code>	Same as <code>cross.attn</code> but adds raw local mean to the MLP input.
<code>transf.concat.narg</code>	Local positions to Transformer encoder; goal g is decoder/query input.
<code>lstm.narg.concat.history</code>	LSTM encodes previous states, tactics, and arguments. Goal attends to history.
<code>transf.concat.narg.history</code>	Transformer encodes hypotheses; goal queries memory/history.

Table 1. Model Descriptions.

In **Phase 1**, we experimented with a no-argument, local, and no history model. We measured class imbalance on LeanDojoML no-argument tactics, then trained `mlp.narg` with

$n = 0$. We also compared original sampling, count capping (5K max, 1K min), and weighted loss

$$\mathcal{L} = -w_y \log p_\theta(y | g), \quad w_y \propto (\text{count}(y) + 1)^{-1/2}.$$

Capping was useful as an ablation but discarded too much data; weighted loss gave the best macro behavior without removing frequent tactics, so later models use it. The same model was tested on different no-argument tactic sets of size 3, 10, and 22, spread across all tactic families where possible. Then in **Phase 2**, we tried a no-argument plus one local argument model without history. We compared using $n = 1$ arg for the *mlp.narg.cross.attn*, *cross.attn.concat*, and *transf.concat.narg* models. In **Phase 3**, we tried a no-argument plus one local argument model with history. We next converted filtered tactic rows into theorem traces and trained the first theorem-level rollout policy. This model is *lstm.narg.concat.history* with $n = 1$. This tests whether accepted first-step behavior (previous phases) compounds when decisions are made sequentially inside a proof. Then in **Phase 4**, we built a model with no-arguments plus n local arguments and history. We then compared *lstm.narg.concat.history* against *transf.concat.narg.history* with $n = 6$ arg. Lastly in **Phase 5**, we built a no-argument plus n local/global arguments with history model. We then compared *lstm.narg.concat.history* against *transf.concat.narg.history* with $n = 16$ arg, also the final dataset expands to 96 tactic kinds and 32 local hypothesis positions and 10k trained global identifiers. If the selected argument class is GLOBAL, a learned query vector is scored against the 10k global-identifier embedding table by cosine similarity. All hyperparameters and filter rules are supplemental.

3. Experiments and Results

3.1. Nazrin GNN

The model (Table 7) trained for ≈ 8 hours on a CPU node of Georgia Tech’s PACE cluster, achieving validation accuracy 0.8533. We applied t-SNE [19] to the final embeddings. Figure 1 shows clean separation by syntactic node type, confirming that the GNN learns structural features of the ExprGraph. Figure 2 shows that tactic-kind clusters overlap uniformly, indicating collapse to dominant classes (intro, apply).

The overlap has two plausible explanations: (a) the model genuinely cannot separate tactic kinds because the same proof state admits multiple valid tactics (e.g., *intro* and *constructor* both apply when the goal is a conjunction); (b) mean aggregation causes oversmoothing, erasing the local structural differences that distinguish tactic kinds. Four ablations would distinguish these: set $\lambda_{\text{arg}} = \lambda_{\text{cs}} = 0$ to focus on kind separation; replace mean aggregation with max; add Supervised Contrastive Loss [12]; or replace focal

loss with label-smoothed cross-entropy [14]. We did not execute these ablations before pivoting because three further issues made the GNN+Mathlib direction unpromising: the six atomic tactics are not formally proven complete; GNNs encode states well but do not natively support the tree search and hypothesis selection that multi-step proving requires; and the 6,635 cases examples are insufficient for sound argument prediction.

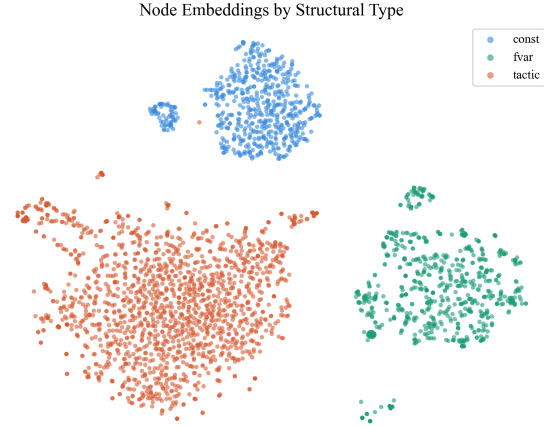


Figure 1. t-SNE of GNN embeddings colored by syntactic node type. Clusters are well-separated, indicating the model encodes structural features of the ExprGraph.

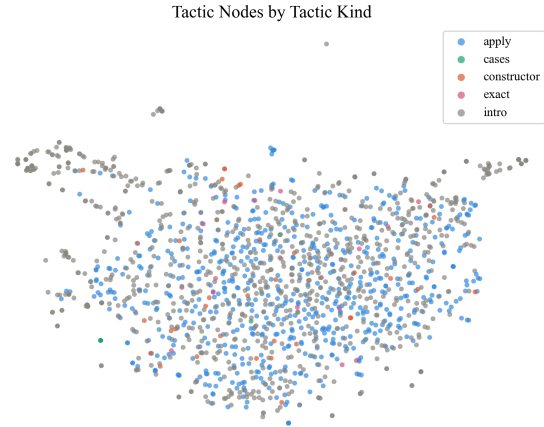


Figure 2. t-SNE of GNN embeddings colored by tactic kind. Clusters overlap uniformly, indicating collapse toward dominant classes (intro, apply). Overlap between *intro* and *constructor* is expected because both tactics apply in nearly identical goal configurations.

3.2. Transformer Tactic Generator

Training was performed on an NVIDIA RTX 4070 Super for 12 epochs; further training produced overfitting on the training set without improving validation similarity. Suc-

cess here is measured by normalized Levenshtein similarity

$$\text{sim}(s, t) = 1 - \frac{\text{lev}(s, t)}{\max(|s|, |t|)},$$

where $\text{lev}(s, t)$ is standard edit distance and 1.0 denotes exact match [2]. This metric is appropriate because (i) it is continuous and gives partial credit to outputs that are structurally close, allowing meaningful ranking even when exact-match accuracy is zero; (ii) Lean tactic strings have well-defined character-level structure, so edit distance captures syntactic proximity; and (iii) it is independent of tokenization choices, enabling comparison across preprocessing pipelines. Exact-match accuracy is the operational gold standard since only an exactly correct tactic is accepted by the Lean kernel, but a model with zero exact matches can still show meaningful learning by producing outputs one or two token edits from valid tactics.

The model achieved a mean Levenshtein similarity of 0.217 over the test set, with $\sim 3\%$ of outputs above 0.5 and none above 0.8 (Table 2). Performance did not vary substantially across tactic types, indicating uniform failure rather than selective difficulty on rare tactics.

Tactic Type	Mean Lev. Similarity
case	0.1924
simp	0.2078
exact	0.2121
rw	0.1954
All	0.2170

Table 2. Transformer Levenshtein similarity by tactic type.

For one goal state, the true tactic was `ext x` while the model predicted `ext <x, y> <x, y> ...` in an unbounded repetition. This is consistent with the exposure bias problem [23]: the decoder is trained on ground-truth prefixes but evaluated on its own predictions, so a single early error shifts the distribution and subsequent tokens reinforce the mistake. On the training set, mean similarity was 0.2362, confirming that the model has limited memorization capacity. Two training examples achieved similarity above 0.9, with one prediction differing from ground truth only in whitespace (Table 9), confirming that Transformer tactic generation is viable at larger scale.

3.3. Transformer Autoinformalization

Training was performed on an NVIDIA A100 for 5 epochs. We measure success here with BLEU [15], consistent with the baselines in [24]. We note that BLEU is an imperfect metric for mathematical prose because any correct paraphrase of a proof is penalized if it differs in surface form from the reference translation. We supplement quantitative results with qualitative examples (Figure 3) and ablations that isolate the contribution of each architectural component.

Table 3 reports BLEU scores for PaLM [7], Codex [6], and our model variants. Our full model achieves a BLEU of 0.0306, substantially below the LLM baselines (22.1 and 43.3); the gap reflects the $\sim 150\times$ parameter disadvantage. However, the ablations are informative: removing feed-forward expansion reduces BLEU from 0.0306 to 0.0263 (-14%); removing RoPE reduces it to 0.0268 (-12%). Removing looping changes the parameter count substantially (Table 3), making the comparison confounded with capacity; the looped model is omitted from relative comparisons for this reason.

The qualitative example in Figure 3 illustrates the primary failure mode: the predicted output introduces an incorrect universally quantified variable ($\forall$ complex z) not present in the formal statement and replaces $\sin(2\pi) = 0$ with the trivially false identity $\sin(z) = \sin(z) \cdot \sin(z)$. This reflects insufficient training data rather than an architectural deficiency: the model has not seen enough formal-informal pairs to learn the algebraic manipulations required to translate even simple trigonometric identities. The repetition failure observed in early training (run-on sentences) was resolved by focal loss and label smoothing.

Model	Parameters	BLEU
PaLM	8×10^9	22.1000
Codex	(undisclosed)	43.3300
Full	7.553×10^7	0.0306
Full w/o Expansion	6.294×10^7	0.0263
Full w/o RoPE	7.606×10^7	0.0268
Full w/o Looping	7.553×10^7	0.0197

Table 3. Informalization Transformer results. BLEU scores for ablations confirm that feed-forward expansion and RoPE each contribute independently. The looped model uses fewer parameters than the full-depth baseline, making direct BLEU comparison partially confounded with capacity.

Formal (Lean 4):

```
theorem Complex.sin_two_pi : sin (2 * \pi) = 0
  ↪ := by simp [two_mul, sin_add]
```

Informal (True):

To prove that $\sin(2\pi) = 0$, we use $2\pi = \pi + \pi$ and the addition formula:

$$\begin{aligned} \sin(2\pi) &= \sin(\pi) \cos(\pi) \\ &\quad + \cos(\pi) \sin(\pi). \end{aligned}$$

Since $\sin(\pi)=0$ and $\cos(\pi) = -1$:

$$\sin(2\pi) = 0$$

Informal (Predicted):

To prove $\sin(2\pi)=0$ for any complex z , we use the definition of sine:

$$\sin(z) = \sin(z) \cdot \sin(z)$$

Figure 3. Example of Herald_proofs informalization.

3.4. Structured Prediction

We report four main metrics: *Full exact* requires the predicted tactic kind and all arguments to match the traced LeanDojo tactic. *Kind exact* measures only tactic-kind accuracy. *Macro accuracy* averages per-tactic accuracy so frequent tactics cannot dominate the score. *Lean acceptance* means the predicted tactic is accepted by Lean from the original proof state.

In **Phase 1**, the loss choice was made on a six-label LeanDojoML no-argument diagnostic: `simp`, `intro`, `rfl`, `constructor`, `ring`, and `omega`. The original sampling had higher ordinary held-out accuracy because it favored frequent `simp` examples, but weighted loss improved macro accuracy from 0.308 to 0.472 where minority tactics were modeled more reliably. As the no-argument action space grows from 3 to 22 tactics, validation exact match falls but checker acceptance rises from 0.140 to 0.222. This is the main reason to report Lean acceptance as prediction can fail to match the Mathlib tactic while still being a useful tactic. Figure 4 shows this acceptance trade-off and the final supervised architecture comparison. In **Phase 2**, the local-argument addition showed that attention helps supervised prediction but not always with checker acceptance: *cross.attn* reached the best validation full exact score (0.572), while the simpler *mlp.narg* produced the best first-step checker acceptance (0.288). This motivated the theorem-level experiments as local prediction is clearly not enough as proof progress depends on whether accepted steps improve over time. Then **Phases 3–5** move to theorem level state learning. The final global-history supervised run shows the strongest large-vocabulary result. *lstm.narg.concat.history* has better validation full exact (0.247), test full exact (0.263), and global-argument accuracy (0.123). *transf.concat.narg.history* has better tactic-kind accuracy (0.623), base-argument (EMPTY, LOCAL, GLOBAL) accuracy (0.516), and macro accuracy (0.518). Both are far above the random global baseline of $1/10K$, but global retrieval remains the main feature to improve. The learning curves also show overfitting therefore checkpoint selection must use validation metrics rather than the final epoch. The overfit curve and checker rollout counts are shown in Figure 4. Checker rollout in Figure 4 *lstm.narg.concat.history* proved less theorems than *transf.concat.narg.history*, all with multi-step proofs in both cases. Also, both models prove one theorem in only 1 tactic where original Mathlib had multiple.

Overall, the best architecture depends on the metric. For supervised full-exact and global argument prediction, the *lstm.narg.concat.history* model is strongest. For theorem proving, *transf.concat.narg.history* achieved better result proving and solving more multi-step traces, while also achieving the best macro score among the final models. The best tactic set is therefore not the smallest no-argument set,

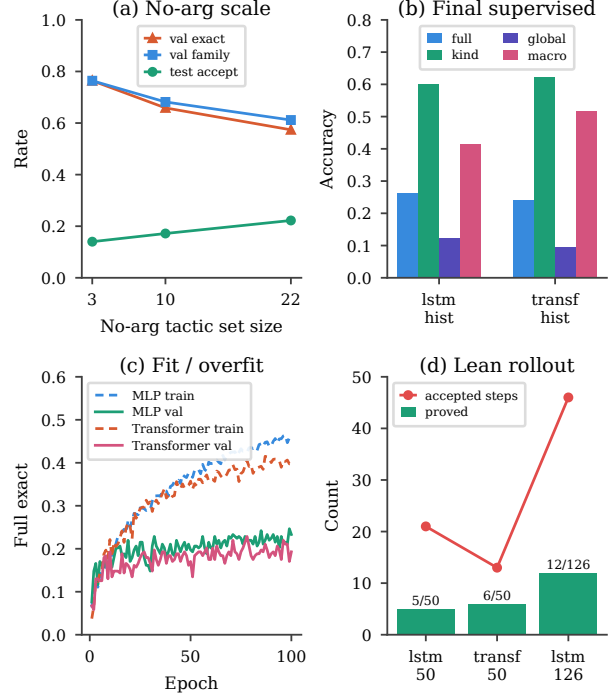


Figure 4. Phases 1 and 5.

even though it has the highest exact accuracy. The final 96-tactic global-history set is necessary for theorem rollout because it covers local and global arguments. Thus our recommended model is *transf.concat.narg.history* with the largest tactic set, selected by validation full exact and evaluated by Lean checker. Additional figures, parameters, etc. are included in the supplemental material.

4. Conclusion

Transformer-based tactic generators can produce syntactically Lean-like output and occasionally achieve near-exact tactic prediction, but they fail to maintain logical consistency over multi-step proofs without kernel feedback. However, the Transformer-based *transf.concat.narg.history* model performs well on proving multi-step theorems, including simplifying proof steps compared to the original theorem. Structural goal embeddings from an AST-GNN provide a useful prior for tactic selection and argument prediction, particularly when combined with attention over local hypotheses. The principal gap in all models is the absence of a feedback loop between generation and verification. Future work should integrate reinforcement learning against the Lean REPL to close this loop and apply the resulting system to Putnam contest problems.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019. 3
- [2] Algorithms and Theory of Computation Handbook. "levenshtein distance". in *Dictionary of Algorithms and Data Structures* [online], Paul E. Black, ed., 1999. 5
- [3] Leni Aniva, Iori Oikawa, David Dill, and Clark Barrett. Nazrin: Atomic tactics for graph neural networks for theorem proving in lean 4, 2026. 1, 2
- [4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. 2
- [5] cat searcher. Leandojo benchmark 4 random, 2024. Hugging Face Dataset. 2, 9
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. July 2021. 2, 5
- [7] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. Apr. 2022. 2, 5
- [8] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, page 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. 1
- [9] Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated lean 4 dataset. In *The Thirteenth International Conference on Learning Representations*, 2025. 2
- [10] Dan Hendrycks and Kevin Gimpel. Bridging nonlinearities and stochastic regularizers with gaussian error linear units. *CoRR*, abs/1606.08415, 2016. 2
- [11] Bharti Khemani, Shruti Patil, Ketan Kotecha, and Sudeep Tanwar. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, 11(1):18, 2024. 2
- [12] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *CoRR*, abs/2004.11362, 2020. 4
- [13] Tsung-Yi Lin, Priya Goyal, Ross B. Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. *CoRR*, abs/1708.02002, 2017. 2
- [14] Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. When does label smoothing help? *Advances in neural information processing systems*, 32, 2019. 4
- [15] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics. 5
- [16] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers, 2025. 3
- [17] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021. 3
- [18] The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, New Orleans, LA, USA, Jan. 2020. ACM. 1
- [19] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. 4
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. 3
- [21] Qingxiang Wang, Cezary Kaliszyk, and Josef Urban. First experiments with neural translation of informal to formal mathematics, 2018. 1
- [22] Qingxiang Wang, Cezary Kaliszyk, and Josef Urban. First experiments with neural translation of informal to formal mathematics, 2018. 2

- [23] Yezhen Wang, Tong Che, Bo Li, Kaitao Song, Hengzhi Pei, Yoshua Bengio, and Dongsheng Li. Your autoregressive generative model can be better if you treat it as an energy-based one. *arXiv preprint arXiv:2206.12840*, 2022. 3, 5
- [24] Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Auto-formalization with large language models. *Advances in neural information processing systems*, 35:32353–32368, 2022. 1, 3, 5
- [25] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models, 2023. 1, 2
- [26] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Graph neural networks: A review of methods and applications. *CoRR*, abs/1812.08434, 2018. 2

5. Appendix

Student	Contributions
Mark Oussoren	Built and trained the Nazrin-inspired GNN model. Implemented the transposing atomization algorithm, the ExprGraph data structure, and Mathlib data extraction in Lean. Analyzed embeddings with t-SNE.
Benjamin Chen	Built and trained the autoinformalization Transformer. Implemented RoPE, looped Transformer layers, and feed-forward expansion. Analyzed model behavior and contributed to visualization strategy.
Diego Franco	Built and trained the models listed in the Structured Predicted section in paper and provided supplemental information. Performed dataset analysis and filters (for theorems).
Rahul Palnitkar	Built and trained the encoder-decoder Transformer for tactic generation. Analyzed model behavior and outputs. Conducted hyperparameter tuning via Optuna.

Table 4. Contributions of team members.

Goal State	Tactic
$a \ b \ c : \mathbb{N}$ $\vdash (a \cdot b + c) \bmod b = c \bmod b$	rw [Nat.mul_comm, Nat.mul_add_mod]

Table 5. Example goal-tactic pair from LeanDojo [5].

Tactic	Count
exact	33,205
intro	276,005
apply	250,810
constructor	71,712
cases	6,635
revert	2

Table 6. Tactic distribution in Mathlib atomization. The four-order-of-magnitude range between `intro` and `revert` motivates inverse-frequency weighted loss and focal loss in all classification heads.

Hyperparameter	Range	Default
Epochs	$\mathbb{N}$	100
Learning Rate	$(0, 1]$	0.001
Batch Size	$\{1, \dots, N\}$	8
Embed Dim d_e	$\mathbb{N}$	32
Hidden Dim d	$\mathbb{N}$	$4 + d_e$
Layers	$\mathbb{N}$	5
Grad Clip	$\mathbb{R}_+$	1.0
LR Schedule	Cosine / Step	Cosine
Aggregation	mean / max / sum	mean
Activation	GELU / ReLU	GELU
Tactic Loss	CE / Focal	Focal
Arg Loss	CE / Focal	CE
CS Loss	MSE / Huber	MSE
$(\lambda_\tau, \lambda_{\text{arg}}, \lambda_{\text{cs}})$	$\mathbb{R}_+^3$	(1, 0.1, 0.01)

Table 7. GNN hyperparameter space and defaults.

Hyperparameter	Range	Optimal
Learning Rate lr	$(10^{-5}, 10^{-3}]$	2.30×10^{-4}
Hidden Dimension d	$\{64, 128, 256, 512\}$	256
Transformer Layers	$[2, 6]$	4
Attention Heads	$\{2, 4, 8\}$	4
Weight Decay	$[0, 0.1]$	0.0552

Table 8. Transformer hyperparameter search space and optimal values found by Optuna TPE over 10 trials of 3 epochs each.

True Tactic	Predicted Tactic
<pre>have hfg_int : ∀ s, <a>MeasurableSet s → μ s < ∞ → <a>MeasureTheory. IntegrableOn (f - g) s μ := fun s hs hμs => (hf_int.finite s hs hμs). <a>MeasureTheory. Integrable.sub (hg_int.finite s hs hμs)</pre>	<pre>have hfg_int : ∀ s , <a > MeasurableSet </ a > s → μ s < ∞ → < a > MeasureTheory . IntegrableOn </ a > (f - g) s μ := fun s hs hμs => (hf_int.finite s hs hμs).< a > MeasureTheory . Integrable . sub </ a > (hg_int.finite s hs hμs)</pre>

Table 9. Near-exact tactic prediction on a training example. The two outputs differ only in whitespace around XML-style tags introduced by the LeanDojo trace format. Levenshtein similarity > 0.9 .